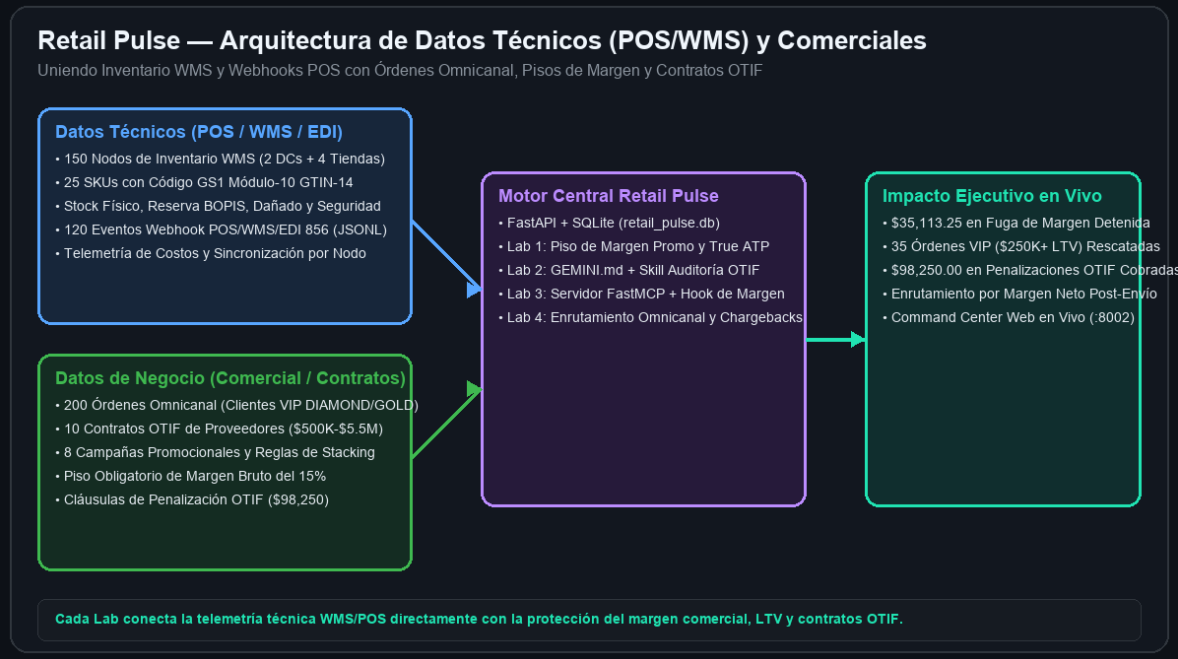
Ingeniería Agéntica con Google Antigravity — Edición Retail: Construyendo 'Retail Pulse'

Enrutamiento Omnicanal, Sincronización True ATP e Inteligencia de Márgenes y Contratos OTIF (Impartido por Tryolabs)



Misión del Taller y Arquitectura de Datos

- **Datos Técnicos WMS/POS:** 150 nodos de inventario (2 DCs + 4 Tiendas), 25 SKUs con GS1 GTIN-14
- **Streams de Webhooks y Escaneos:** 120 eventos POS/WMS/EDI 856 en JSONL con bloqueos BOPIS y merma
- **Datos Comerciales:** 200 órdenes omnicanal (LTV Clientes VIP DIAMOND/GOLD, precio vs costo)
- **Contratos de Proveedores OTIF:** 10 acuerdos comerciales ($500K–$5.5M, metas OTIF y penalizaciones)
- **Impacto Medible:** Detener $35,113.25 de fuga de margen, rescatar 35 órdenes VIP y cobrar $98,250 OTIF

Ruta Práctica de 4 Horas: Del Triage a Sistemas Multi-Agente ADK y A2A

Cuatro cápsulas teóricas de 6 minutos seguidas de cuatro laboratorios progresivos de 52–55 minutos



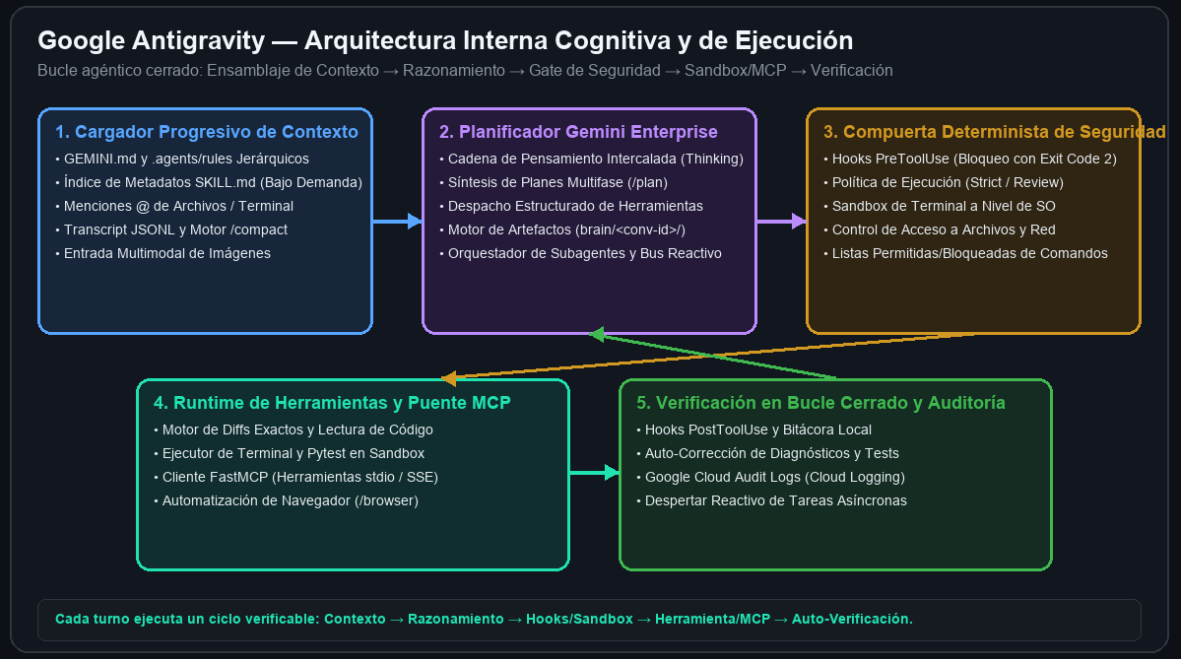
Cronograma Minuto a Minuto

- **Hora 1 (00:00–00:58):** [6m Teoría] Arquitectura Interna y Superficies → [52m Lab 1] Triage y Recuperación Financiera
- **Hora 2 (00:58–01:58):** [6m Teoría] Rules y Skills Progresivos → [54m Lab 2] Estándares de Dominio y Skill Ejecutivo
- **Hora 3 (01:58–02:59):** [6m Teoría] MCP, Hooks y Gobernanza Cloud → [55m Lab 3] Servidor FastMCP Local y Hooks
- **Hora 4 (02:59–04:00):** [6m Teoría] Subagentes, ADK, A2A y Terraform → [55m Lab 4] Capstone Multi-Agente, ADK Dev UI y Cloud
- **Checkpoints Auto-Recuperables:** ¡Ejecutar ./labs/lab-0X/install.sh aplica las soluciones previas automáticamente!

Cada laboratorio incluye LAB_0X_INSTRUCTIONS_ES.pdf, README.es.md, PROMPTS.es.md, install.sh y verify.sh.

Cómo Funciona Google Antigravity Internamente

Dentro del planificador cognitivo, ensamblador de contexto, compuerta de seguridad y runtime en sandbox



El Ciclo Interno de Ejecución en 5 Etapas

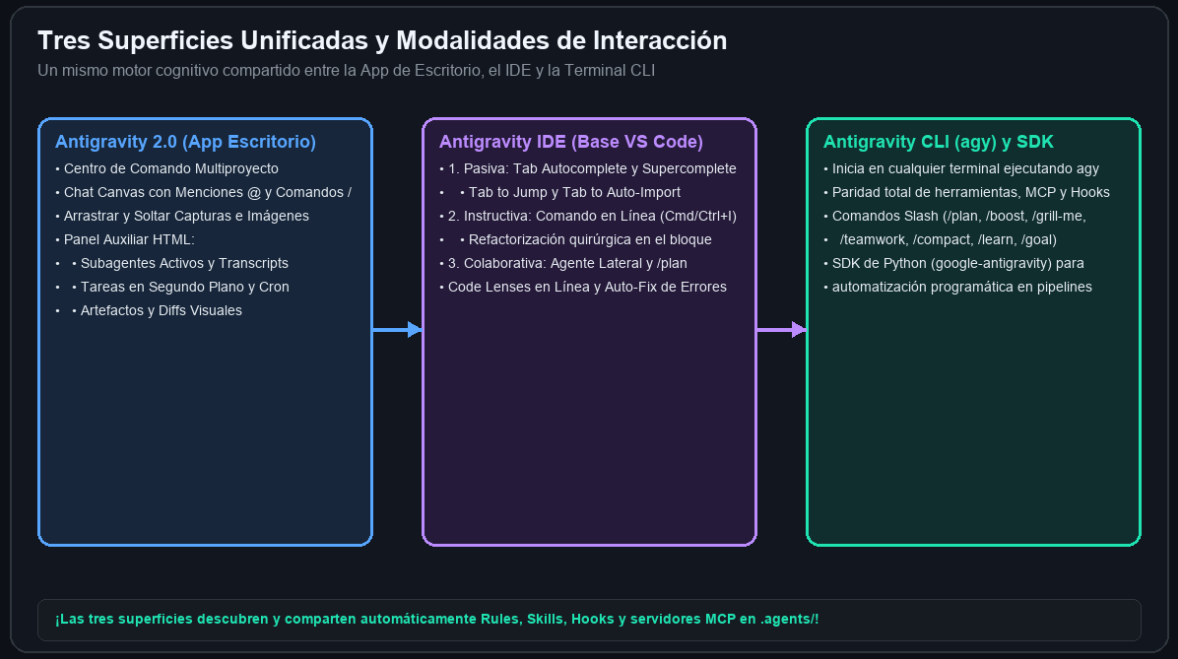
- **1. Cargador Progresivo de Contexto:** Recorre el árbol de directorios buscando GEMINI.md y .agents/rules, indexa metadatos YAML de SKILL.md, resuelve menciones @ y gestiona transcript.jsonl
- **2. Planificador Gemini Enterprise:** Genera razonamiento intercalado (thinking), sintetiza planes (/plan) y despacha herramientas o subagentes en paralelo
- **3. Compuerta Determinista de Seguridad:** Evalúa hooks PreToolUse (código 2 bloquea la acción), políticas de ejecución, ACLs y sandbox de terminal
- **4. Runtime de Herramientas y Puente MCP:** Aplica diffs exactos, ejecuta pytest en sandbox e invoca herramientas FastMCP por stdio/SSE
- **5. Verificación en Bucle Cerrado:** Ejecuta hooks PostToolUse, analiza resultados de tests y se autocorrige hasta pasar todas las pruebas

Concepto Clave: Antigravity no es solo un chat, sino un motor de ingeniería en bucle cerrado verificable. | **Slide 3/15**

tryo•labs

Antigravity 2.0, IDE y CLI (`agy`) — Modalidades Clave

Seleccionando la modalidad y comando slash ideal para cada tarea de ingeniería



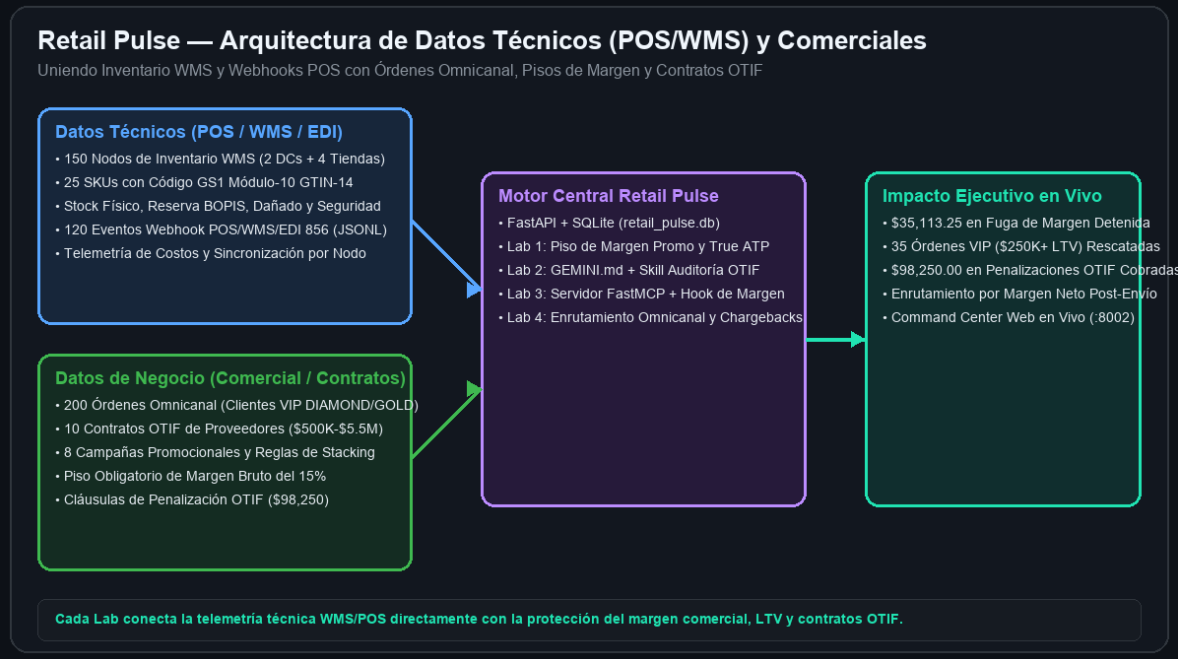
Funcionalidades que Explorarás Hoy

- **Modalidad Pasiva (IDE Tab):** Supercomplete sensible al contexto, Tab-to-Jump entre archivos y Tab-to-Auto-Import
- **Modalidad Instructiva (Cmd/Ctrl+I y Code Lenses):** Selecciona un bloque para refactorización quirúrgica o haz clic en 'Write Tests'
- **Modalidad Colaborativa (Chat Canvas y CLI `agy`):** Razonamiento multi-archivo con @archivo, @carpeta, @terminal y capturas de pantalla
- **Comandos Slash de Alto Impacto:** /plan (arquitectura paso a paso), /grill-me (entrevista de diseño), /boost, /teamwork, /learn y /compact
- **Panel Auxiliar HTML:** Inspección en vivo de Subagentes, Tareas en Segundo Plano, Artefactos y Diffs de Archivos

Tip Pro: Inicia tareas complejas con /plan y menciones @archivo; usa Cmd+I para ajustes puntuales de fórmulas.

Lab 1: Piso de Margen y True ATP — Objetivos y Arquitectura

52 Minutos • Guía PDF Imprimible: labs/lab-01-margin-inventory-triage/LAB_01_INSTRUCTIONS_ES.pdf



Objetivos del Lab 1 y Qué Construirás

- **Objetivo 1 (Triage Agéntico Multi-Archivo):** Dominar `@menciones`, `/plan` y edición en línea `Cmd/Ctrl+I` correlacionando eventos POS/WMS (`pos_wms_webhooks.jsonl`) con código de precios e inventario.
- **Objetivo 2 (Recuperación de Margen y Órdenes VIP):** Detener `$35,113.25 USD` en fuga de margen negativo en 30 órdenes y detectar 35 órdenes VIP `DIAMOND`/`GOLD` (`>$250K` LTV) con stock fantasma.
- **Qué Construirás #1 (Motor de Precios GS1 y Piso de Margen):** Refactorizarás `src/services/promo_pricing.py` validando `gtin_14` Módulo-10 GS1, `Decimal(ROUND_HALF_UP)`, descuentos acumulables multiplicativos y piso mínimo de margen (`15%`).
- **Qué Construirás #2 (Motor de Inventario True ATP):** Refactorizarás `src/services/inventory_atp.py` calculando `raw_atp = on_hand - reserved - safety_stock - unprocessed_webhook_delta_qty`.
- **Entregable y Verificación:** Pasar los 5 tests en `tests/test_lab01_triage.py` (`./labs/lab-01-margin-inventory-triage/verify.sh`) y validar en `http://localhost:8002`.

tryo•labs

Instrucciones del Lab 1 y Prompts para Copiar y Pegar

Sigue los Pasos 1–5 a la izquierda y copia los prompts desde labs/lab-01-margin-inventory-triage/PROMPTS.es.md

Instrucciones Paso a Paso del Lab 1

- **Paso 1 (Inicializar Lab 01):** Ejecuta `./labs/lab-01-margin-inventory-triage/install.sh` para inicializar `data/retail_pulse.db`.
- **Paso 2 (Reproducir Fallos):** Ejecuta `./.venv/bin/pytest tests/test_lab01_triage.py` para observar los fallos de margen y ATP.
- **Paso 3 (Corregir `src/services/promo_pricing.py`):** Valida `gtin_14` Módulo-10 GS1, usa `Decimal(ROUND_HALF_UP)`, no combines cupones `EXCLUSIVE`, aplica los acumulables multiplicativamente y protege `contract_min_margin_floor_pct`.
- **Paso 4 (Corregir `src/services/inventory_atp.py`):** Calcula `raw_atp = on_hand - reserved - safety_stock - unprocessed_webhook_delta_qty` y `true_atp = max(0, raw_atp)`.
- **Paso 5 (Verificar):** Ejecuta `./labs/lab-01-margin-inventory-triage/verify.sh` (`5 passed`: `$35,113.25` recuperados; 35 órdenes VIP detectadas).

Prompts Optimizados (PROMPTS.es.md) y Verificación

- **Prompt 1 — Diagnóstico (@Menciones):** '@src/services/promo_pricing.py @src/services/inventory_atp.py @data/logs/pos_wms_webhooks.jsonl @tests/test_lab01_triage.py Ejecuta pytest y explica por qué 30 órdenes pierden margen y 35 órdenes VIP tienen inventario fantasma.'
- **Prompt 2 — Planificación (/plan):** '/plan Crea un plan de corrección para promo_pricing.py (GS1 GTIN-14, Decimal, promos acumulables multiplicativas, piso de margen) e inventory_atp.py (restar reserved, safety_stock y webhook delta).'
- **Prompt 3 — Edición en Línea (Cmd+I):** Selecciona `calculate_node_atp` en `inventory_atp.py` -> 'Resta reserved_qty, safety_stock_qty y unprocessed_webhook_delta_qty de on_hand_qty.'
- **Prompt 4 — Verificación Autónoma:** 'Aplica las correcciones en @src/services/promo_pricing.py y @src/services/inventory_atp.py y ejecuta ./labs/lab-01-margin-inventory-triage/verify.sh.'

tryo•labs

Cómo Funcionan Internamente las Rules y Skills Progresivos

Especializando a Antigravity en tu industria sin saturar la ventana de contexto



Rules vs. Skills vs. /learn

- **Reglas Jerárquicas (GEMINI.md y .agents/rules/*.md):** Invariantes de dominio cargados desde la raíz del workspace hasta los subdirectorios (deduplicados automáticamente)
- **Skills con Divulgación Progresiva (.agents/skills/<name>/SKILL.md):** Solo el frontmatter YAML (name, description) entra al prompt inicial; el cuerpo completo y scripts/ se cargan SOLO al activarse
- **Memoria Auto-Mejorable (/learn):** Pide a Antigravity codificar un patrón aprendido durante el debugging en una regla o skill permanente
- **Jerarquía de Prioridad:** Workspace (.agents/) > JSON Declarado (skills.json) > Global (~/.gemini/config/) > Integrado

Regla Práctica: Coloca invariantes permanentes en GEMINI.md/rules y flujos multifase con scripts en SKILL.md.

tryo•labs

Lab 2: Rules de Workspace y Skills Progresivos — Objetivos y Diseño

54 Minutos • Guía PDF Imprimible: labs/lab-02-rules-and-skills/LAB_02_INSTRUCTIONS_ES.pdf



Objetivos del Lab 2 y Qué Construirás

- **Objetivo 1 (Reglas Jerárquicas de Dominio):** Codificar invariantes permanentes GS1 GTIN-14, True ATP, promociones multiplicativas y cargos OTIF en el workspace.
- **Objetivo 2 (Skills con Divulgación Progresiva):** Construir un skill ejecutivo de auditoría comercial y logística en `.agents/skills/` usando frontmatter YAML ligero.
- **Qué Construirás #1 (`GEMINI.md` y `.agents/rules/retail-standards.md`):** Memoria y reglas del workspace exigiendo `Decimal(ROUND_HALF_UP)`, dígito verificador GS1 Módulo-10, piso de margen `15%` y fórmulas OTIF.
- **Qué Construirás #2 (`.agents/skills/margin-otif-audit/SKILL.md`):** Skill progresivo (`name: margin-otif-audit`) que ejecuta `scripts/run_otif_margin_audit.py`.
- **Entregable y Verificación:** Generar `data/exports/executive_margin_otif_audit.json` y `.md` (`$98,250.00` en cargos OTIF) y pasar `./labs/lab-02-rules-and-skills/verify.sh`.

tryo•labs

Instrucciones del Lab 2 y Prompts para Copiar y Pegar

Sigue los Pasos 1–5 a la izquierda y copia los prompts desde labs/lab-02-rules-and-skills/PROMPTS.es.md

Instrucciones Paso a Paso del Lab 2

- **Paso 1 (Inicializar Lab 02):** Ejecuta `./labs/lab-02-rules-and-skills/install.sh` (aplica Lab 01 automáticamente).
- **Paso 2 (Crear `GEMINI.md` en la raíz):** Documenta `sku_catalog`, `wms_inventory`, `vendor_contracts`, `promo_campaigns`, `omnichannel_orders` y las reglas `Decimal(ROUND_HALF_UP)` y `gtin_14`.
- **Paso 3 (Crear `.agents/rules/retail-standards.md`):** Codifica validación GS1 GTIN-14 Módulo-10, fórmula True ATP, descuentos multiplicativos, piso de margen y penalidades OTIF.
- **Paso 4 (Crear `.agents/skills/margin-otif-audit/SKILL.md`):** Incluye frontmatter YAML (`name: margin-otif-audit`, `description: ...`) e instrucciones para correr `./.venv/bin/python scripts/run_otif_margin_audit.py`.
- **Paso 5 (Verificar):** Ejecuta `./labs/lab-02-rules-and-skills/verify.sh` (`5 passed` + reportes ejecutivos JSON/MD).

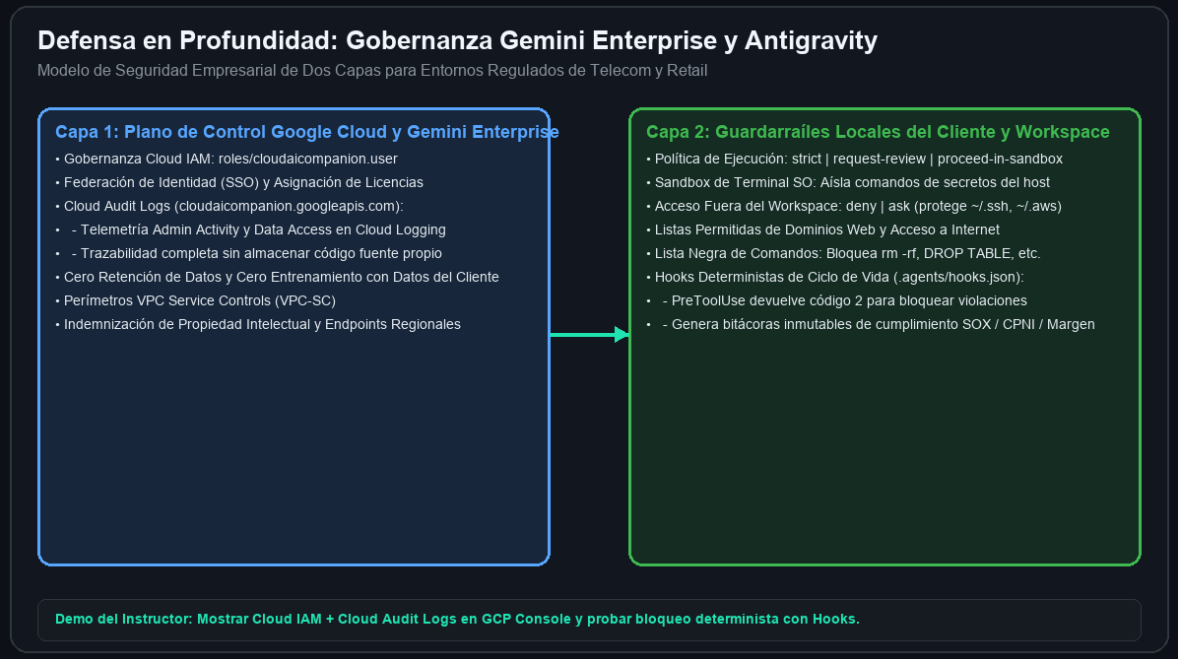
Prompts Optimizados (PROMPTS.es.md) y Verificación

- **Prompt 1 — Crear Rules de Dominio:** '/plan Crea @GEMINI.md y @.agents/rules/retail-standards.md exigiendo Decimal ROUND_HALF_UP, validación GS1 GTIN-14 Módulo-10, deducción de webhook delta en True ATP, promos multiplicativas y reglas OTIF.'
- **Prompt 2 — Crear Skill Progresivo:** 'Crea @.agents/skills/margin-otif-audit/SKILL.md con frontmatter YAML (name: margin-otif-audit) que ejecute ./scripts/run_otif_margin_audit.py y genere data/exports/executive_margin_otif_audit.json y .md.'
- **Prompt 3 — Invocar el Skill:** 'Ejecuta una auditoría ejecutiva de márgenes, True ATP y penalidades OTIF de proveedores usando nuestro skill del workspace.'
- **Comando de Verificación:** Ejecuta `./labs/lab-02-rules-and-skills/verify.sh` y prueba `/learn`.

tryo•labs

Puente MCP en Vivo, Hooks Deterministas y Gobernanza Gemini Enterprise

Conectando bases de datos en tiempo real con seguridad empresarial demostrable en Google Cloud



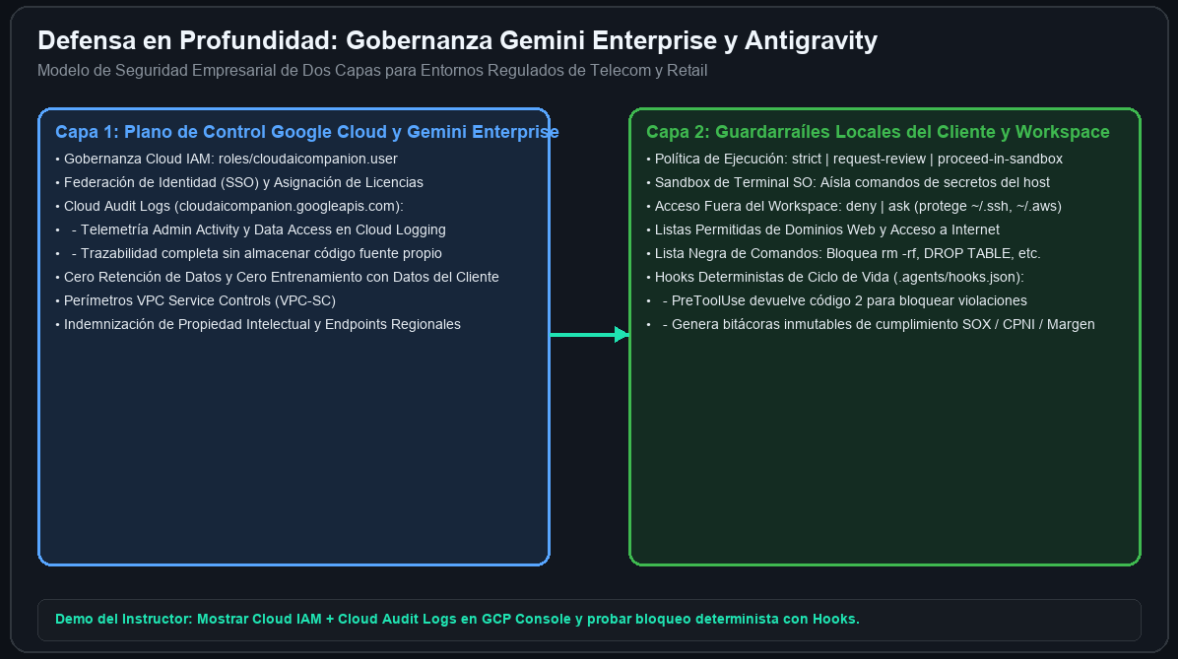
Seguridad y Gobernanza en 2 Capas

- **Capa 1 — Google Cloud y Gemini Enterprise (Demo en Vivo del Instructor):** Cloud IAM (roles/cloudaicompanion.user), Cloud Audit Logs (cloudaicompanion.googleapis.com), perímetros VPC-SC, Cero Retención y Cero Entrenamiento con código del cliente
- **Capa 2 — Seguridad Local en Antigravity:** Política de Ejecución (strict / request-review / proceed-in-sandbox), Sandbox de Terminal SO, bloqueo de archivos fuera del workspace y listas permitidas/bloqueadas
- **Contexto en Vivo vía FastMCP (.agents/mcp_config.json):** Expone herramientas Python tipadas sobre SQLite y telemetría vía stdio/SSE
- **Hooks Deterministas (.agents/hooks.json):** El script PreToolUse inspecciona el JSON en stdin; devolver código 2 bloquea violaciones e imprime un registro de auditoría inmutable

Demo del Instructor: Mostrar Cloud Audit Logs en GCP Console + disparar un bloqueo PreToolUse con código 2 localmente. | **Slide 10/15**

Lab 3: Servidor FastMCP Retail y Hooks de Margen — Objetivos y Diseño

55 Minutos • Guía PDF Imprimible: labs/lab-03-mcp-and-hooks/LAB_03_INSTRUCTIONS_ES.pdf



Objetivos del Lab 3 y Qué Construirás

- **Objetivo 1 (Conexión Omnicanal en Vivo vía MCP):** Conectar Antigravity a tablas SQLite de inventario WMS, órdenes VIP y contratos OTIF mediante un servidor `FastMCP`.
- **Objetivo 2 (Guardarraíles Deterministas de Margen):** Interceptar ediciones de código y comandos shell con un hook `PreToolUse` que bloquee con código `2` cualquier intento de desactivar el piso de margen (`15%`).
- **Qué Construirás #1 (`mcp_server/retail_mcp.py` y `.agents/mcp_config.json`):** 4 herramientas `@mcp.tool()` (`query_true_atp_inventory`, `audit_vendor_otif_contracts`, `simulate_promo_margin_guard`, `list_at_risk_vip_orders`).
- **Qué Construirás #2 (`.agents/hooks.json` y `scripts/margin_guard_hook.py`):** Hook determinista `PreToolUse` que bloquea `enforce_floor=False`, margen `< 0.15` o SQL `DROP`/`DELETE`, auditando en `data/logs/hook_compliance_audit.jsonl`.
- **Entregable y Verificación:** Pasar los 5 tests de MCP y Hooks en `./labs/lab-03-mcp-and-hooks/verify.sh`.

tryo•labs

Instrucciones del Lab 3 y Prompts para Copiar y Pegar

Sigue los Pasos 1–5 a la izquierda y copia los prompts desde labs/lab-03-mcp-and-hooks/PROMPTS.es.md

Instrucciones Paso a Paso del Lab 3

- **Paso 1 (Inicializar Lab 03):** Ejecuta `./labs/lab-03-mcp-and-hooks/install.sh` (aplica Labs 01–02 automáticamente).
- **Paso 2 (Completar `mcp_server/retail_mcp.py`):** Implementa `simulate_promo_margin_guard(promo_code, category, loyalty_tier)` y `list_at_risk_vip_orders(loyalty_tier, limit)`.
- **Paso 3 (Registrar `.agents/mcp_config.json`):** Registra `retail-pulse-mcp` ejecutando `.venv/bin/python -m mcp_server.retail_mcp`.
- **Paso 4 (Configurar `.agents/hooks.json`):** Conecta `PreToolUse` a `scripts/margin_guard_hook.py` (bloquea `enforce_floor=False` o margen `< 0.15` con código `2`).
- **Paso 5 (Verificar):** Ejecuta `./labs/lab-03-mcp-and-hooks/verify.sh` (`5 passed` + bitácora de auditoría).

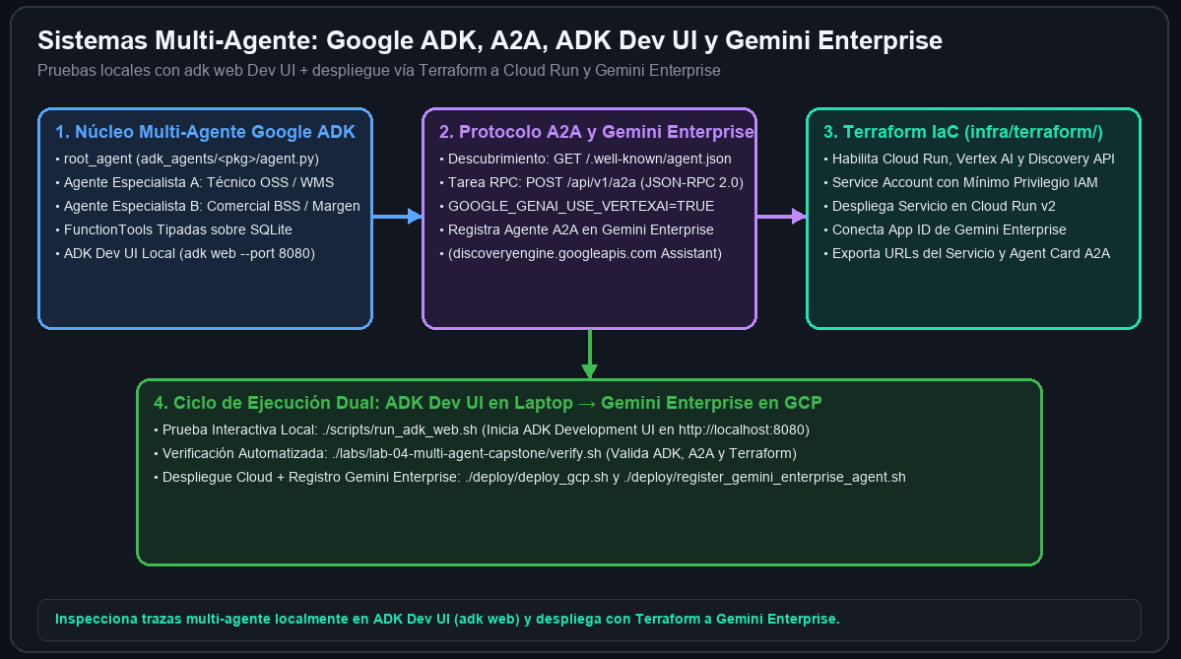
Prompts Optimizados (PROMPTS.es.md) y Verificación

- **Prompt 1 — Completar Servidor FastMCP:** '/plan Implementa simulate_promo_margin_guard y list_at_risk_vip_orders en @mcp_server/retail_mcp.py cumpliendo con @tests/test_lab03_mcp_hooks.py.'
- **Prompt 2 — Configurar MCP y Hooks:** 'Crea @.agents/mcp_config.json para retail-pulse-mcp y @.agents/hooks.json conectando PreToolUse a ./scripts/margin_guard_hook.py.'
- **Prompt 3 — Probar Bloqueo del Hook (Código 2):** Ejecuta `echo '{"tool_name":"edit_file","tool_input":{"Path":"src/services/promo_pricing.py","Replacement":"enforce_floor=False"}}' | .venv/bin/python scripts/margin_guard_hook.py`
- **Comando de Verificación:** Ejecuta `./labs/lab-03-mcp-and-hooks/verify.sh` y revisa `data/logs/hook_compliance_audit.jsonl`.

tryo•labs

Sistemas Multi-Agente con Google ADK, Protocolo A2A y Terraform

Diseñando arquitecturas multi-agente localmente con `adk web` y desplegando a Cloud Run y Gemini Enterprise

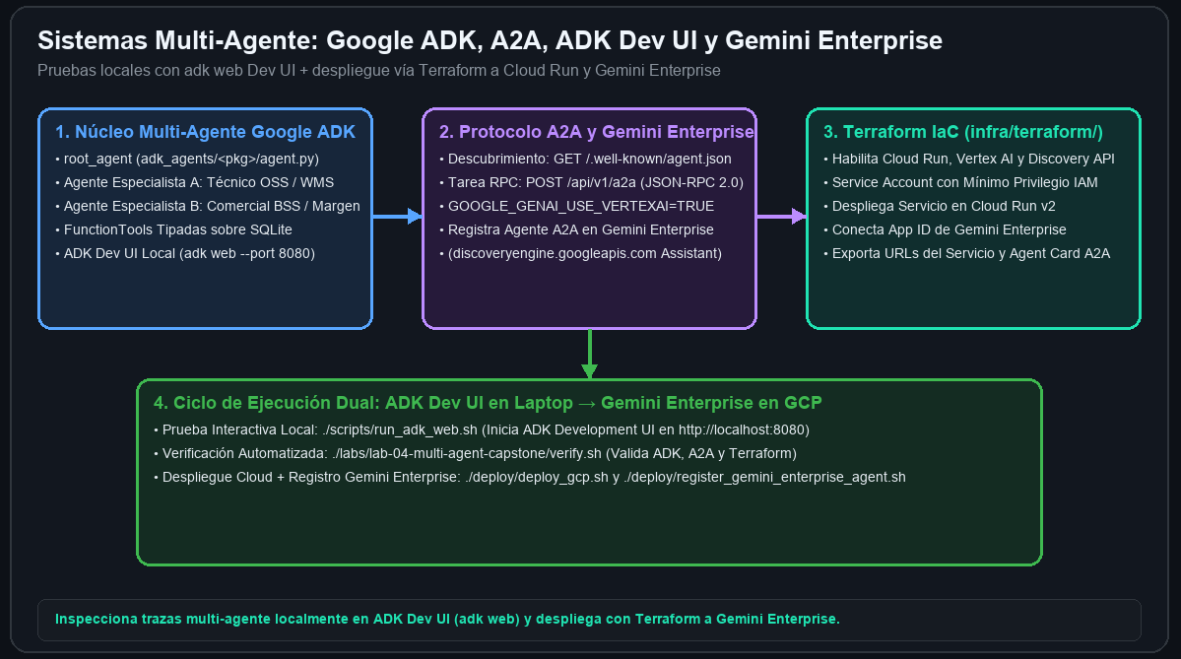


Arquitectura Multi-Agente de Punta a Punta

- **Subagentes de Antigravity (/teamwork, /boost, /grill-me):** Delega tareas paralelas de investigación y programación dentro de Antigravity
- **Google ADK (Agent Development Kit):** Construye jerarquías multi-agente de dominio (`root_agent` coordinando agentes especialistas Técnico y Comercial con herramientas tipadas)
- **Pruebas Locales con ADK Dev UI (`adk web`):** Ejecuta `./scripts/run_adk_web.sh` (http://localhost:8080) para inspeccionar trazas de eventos multi-agente, llamadas a herramientas y estado
- **Protocolo A2A (Agent2Agent):** Publica `GET /.well-known/agent.json` (Agent Card) y `POST /api/v1/a2a` (JSON-RPC 2.0 `tasks/send`) para interoperabilidad entre agentes
- **Terraform + Gemini Enterprise:** Aprovisiona Cloud Run v2 e IAM vía `infra/terraform/` (`deploy_gcp.sh`) y registra tu agente A2A en Gemini Enterprise (`register_gemini_enterprise_agent.sh`)

Lab 4 Capstone: Sistemas Multi-Agente ADK, A2A y Terraform — Objetivos

55 Minutos • Guía PDF Imprimible: labs/lab-04-multi-agent-capstone/LAB_04_INSTRUCTIONS_ES.pdf



Objetivos del Lab 4 y Qué Construirás

- **Objetivo 1 (Diseño Multi-Agente con Google ADK y A2A):** Construir un sistema jerárquico multi-agente con Google ADK (`LlmAgent`) y exponer su Agent Card + endpoint JSON-RPC 2.0 A2A.
- **Objetivo 2 (Prueba Local en `adk web` y Despliegue Cloud con Terraform):** Probar delegaciones entre agentes en `adk web` (`http://localhost:8080`) y validar el despliegue a Cloud Run + Gemini Enterprise con Terraform.
- **Qué Construirás #1 (`src/services/fulfillment_router.py`):** Optimizador omnicanal de re-enrutamiento (`evaluate_candidates` y `optimize_at_risk_orders`) rescatando las 35 órdenes VIP según margen neto post-envío.
- Qué Construirás #2 (`src/agents/adk_a2a_orchestrator.py` y `adk_agents/retail_pulse/`): `WMSInventoryATPAgent` + `MerchandisingMarginOTIFAgent` coordinados por `RetailPulseOrchestratorAgent`, más `GET /.well-known/agent.json` y `POST /api/v1/a2a`.
- **Qué Construirás #3 (Infraestructura Cloud):** Validar `infra/terraform/main.tf` y el registro en Gemini Enterprise con `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run`.

Instrucciones del Lab 4, Prueba en `adk web` y Despliegue Terraform

Sigue los Pasos 1–5 a la izquierda y copia los prompts desde labs/lab-04-multi-agent-capstone/PROMPTS.es.md

Instrucciones Paso a Paso del Lab 4

- **Paso 1 (Inicializar Lab 04):** Ejecuta `./labs/lab-04-multi-agent-capstone/install.sh`.
- **Paso 2 (Enrutador Omnicanal):** Implementa `evaluate_candidates()` y `optimize_at_risk_orders()` en `src/services/fulfillment_router.py` (re-enrutando las 35 órdenes VIP según mayor margen neto post-envío).
- **Paso 3 (Orquestador Google ADK y A2A):** Completa `src/agents/adk_a2a_orchestrator.py` (`WMSInventoryATPAgent`, `MerchandisingMarginOTIFAgent`, `RetailPulseOrchestratorAgent`, `GET /.well-known/agent.json`, `POST /api/v1/a2a`).
- **Paso 4 (Interfaz Local `adk web`):** Ejecuta `./scripts/run_adk_web.sh --port 8080` y prueba `retail_pulse` en `http://localhost:8080`.
- **Paso 5 (Terraform y Verificación Total):** Ejecuta `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run` y `./labs/lab-04-multi-agent-capstone/verify.sh` (**¡20/20 tests pasando!**).

Prompts Optimizados, `adk web` y Comandos Cloud

- **Prompt 1 — Enrutador Omnicanal:** '/plan Implementa evaluate_candidates y optimize_at_risk_orders en @src/services/fulfillment_router.py ordenando nodos por (-post_shipping_net_margin_usd, shipping_cost_usd, prioridad DC).'
- **Prompt 2 — Orquestador ADK + A2A:** '/teamwork Implementa WMSInventoryATPAgent, MerchandisingMarginOTIFAgent, RetailPulseOrchestratorAgent, build_a2a_agent_card y handle_a2a_jsonrpc en @src/agents/adk_a2a_orchestrator.py.'
- **Prompt 3 — Interactivo en `adk web` (http://localhost:8080):** 'Ejecuta remediación omnicanal: aplica el piso de 15% de margen, re-enruta las 35 órdenes VIP y reclama penalidades OTIF.'
- **Despliegue y Verificación:** Ejecuta `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run` y `./labs/lab-04-multi-agent-capstone/verify.sh`.